

AI 编程范式下产品开发全过程

termio.sh · Terminal-first Agentic Development Environment

袁继伟

少个分号 · 40–60 min

<https://termio.sh>

1. 序章—Demo 演示
2. 缘起—问题驱动：为什么做 Terminal-first ADE?
3. 产品设计—用户画像、核心场景、MVP
4. 技术实现—架构总览 + 三个关键难点
5. **Agentic Engineering** —skills · workflow
6. 迭代优化—从可用到好用
7. 未来展望
8. 总结与启示

序章：Demo 演示

先看产品，再听故事。

段	看什么	证明什么
1	Claude / Codex / OpenCode 并排	多 agent 编排，不是又一个 IDE
2	侧栏状态点 + 菜单栏 tray	idle / working / needsAttention / done
3	iPhone 扫码续会话	本地 PTY host + companion

Stop babysitting terminals.

在真实原生终端里，编排你已经在用的 *coding agent*。

- 不是 Cursor 插件 · 不是 Electron 套壳 · 不是云端托管 agent
- 是：macOS 原生 · libghostty · 本地 PTY · 可选 iOS 远程

Agent 时代，最慢的一环变了：

1. 窗口 **babysitting** 一十几个 terminal，不知道谁在跑
2. 状态不可见—思考中还是卡权限？
3. 工具碎片—Claude Code / Codex / OpenCode / Pi ...
4. 桌面 ↔ 手机断层

问题不是「不会写代码」，是 编排与注意力。

为什么是 Terminal-first?

- Coding agent 的「家」天然在终端（CLI 已是主流）
- TUI 就是 agent 的 UI ——再包一层 Web IDE 是重复劳动
- 终端 = 真实 PTY = 真实权限 / shell / 工具链

IDE 帮人写代码；**ADE** 帮人管 agent。

做

- 原生 macOS (Swift)
- 拥有 PTY (libghostty)
- 多 agent + 状态 tray
- 本地优先、无账号
- 配置驱动适配 agent

不做

- Electron / 纯 Web 壳
- 只 tail 日志、不拥有运行时
- 做成完整 IDE
- 强制云同步 / 账号门禁
- 每加 agent 改一堆 Swift

同桌面: Unpeel · cmux · Conductor · herdr · Warp Oz ...

termio 押注的差异

1. 原生 + libghostty + 零配置状态 + 菜单栏 ambient
2. first-class worktree / 会话编排
3. iOS companion (监督面, 不是第二套 TUI)
4. 小而意见化——不和 cmux 拼 breadth

用户画像（只打一个核心）

重度 CLI agent 用户

- 日常用 Claude Code / Codex / OpenCode
- 多 session 并行，靠自己记「谁在干活」
- 在乎：本地、隐私、原生手感、键盘流
- 不在乎：再学一个云 IDE、再开一个账号

三个场景

1. 启动与并行 (project → multi-session)
2. 盯状态 (侧栏 + tray)
3. 离开工位仍在线 (iOS companion)

MVP 必须有：侧栏 · 真 PTY · agent 预设 · 状态可见 · 无账号
故意后置：第二个 Cursor · never-die host · 过早平台化

关键产品决策（真实做过的）

- 2026-06-26: Zed fork 推倒重来 → Swift + libghostty
- Agent = 配置文件，不是枚举（加 Grok 曾改 6 个文件）
- 状态三层：OSC/铃 → hooks → title 规则
- Git 面板做到 TextKit 级 diff（对照 Desktop / JetBrains）
- iOS：有终端，但不把 TUI 当唯一监督面

```
macOS termio (Swift + AppKit/SwiftUI)
  Sidebar · TermioStore · MenuBar · Git · Editor
  |
  libghostty (Metal) <-> PTY <-> shell / agent CLI
  HookListener <- termio agent report (unix socket)
  AgentCatalog <- Resources/agents/*.json + ~/.termio/
  CompanionServer (WebSocket)
    | TermioShared.WireProtocol
    v
  iOS TermioMobile Landing / Docs (Next.js · R2)
```

难点 1: 终端设计

目标：原生、快、会话可切换、agent TUI 全屏可用。

- libghostty (Metal), 非 WebView
- SurfaceCache: 切换 session 重挂 surface, 不重 spawn
- PTY 生命周期 $\times$ focus / resize

近三周真实坑：窄 grid banner 冻住 · resize 不 reflow · focus 丢失

难点 2: Agent 适配

抽象 AgentDefinition: id · command · resume · icon · hook/title rules

同一加载路径: bundled JSON + 用户 manifest

状态: idle → working → needsAttention | done

权威: hooks · OSC 铃 · title 规则 (仲裁)

public contract

termio agent report <working|attention|done|idle>

channel-scoped socket ——dev 不能盖写 release hooks

难点 3: iOS 移动端

- Mac = **PTY host** (唯一真相源)
- iOS = companion client
- WireProtocol: auth · attach · start · resize · files · trace

近三周: Chats/Projects · FAB · Markdown 同步 · renderer 稳定性 · TestFlight

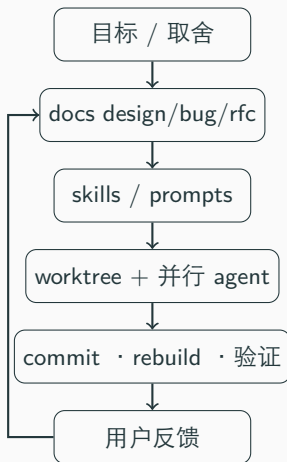
手机需要终端能力, 但不该把 **agent TUI** 当唯一 UI。

三个难点的共性

难点	核心矛盾	解法
终端	真 PTY vs 易嵌入	拥有 surface + 修上游
Agent	CLI 碎片 vs 统一体验	Manifest + hook + 多层状态
iOS	远程可见 vs 本地真相	Mac host + 薄 wire

原则：本地优先 · 零配置兜

底 · 配置驱动 · 先可用再精确



Skills (仓库里真实存在)

Skill	干什么
macos-rebuild-dev	杀进程 → 构建 → 拉起 dev app
ios-rebuild-dev	装真机 / 模拟器
bump-version	打 tag 发版
conventional-commit	规范提交
doc	设计文档 + wiki 索引
app-screenshot-debug	点 UI + 截窗

高频易错步骤 → skill; 一次性

探索 → 普通对话

值得固化

- 构建与重装路径 (Mac / iOS)
- 发版、commit 规范
- 设计决策与事故 handoff
- 带验收标准的任务

不要过度自动化

- 还在摇摆的产品取舍 · UI 审美判断 · 证书类一次性操作

AI 加速回路；判断仍在人。

迭代：近三周真实主题

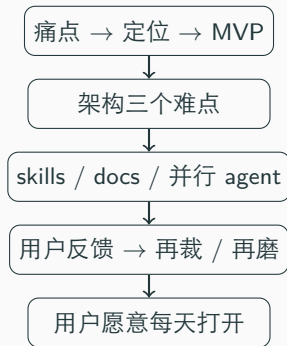
主题	信号	演进
Agent 状态 /clear	转完还在转 / 黄点绿点 trace 旧数据	hooks · title · done conversation rotation
Git	不像 Desktop/WebStorm	TextKit diff
Agent 扩展	加 agent 改多文件	agents/*.json
iOS	FAB · TestFlight	companion parity
终端细节	换行 · focus · reflow	布局时序 + 上游

1. 状态可信是 ADE 的生命线
2. 原生手感是付费意愿的前置
3. 移动端是监督，不是缩小版桌面
4. 删掉错误抽象也是进度（如自研 sandbox）
5. 文档是异步团队协议（一人 + 多 agent 也需要）

未来展望（可讨论）

1. 瓶颈：写代码 → 编排与确认
2. ADE 与 IDE 长期并存
3. 拥有运行时的人赢监督
4. 本地 + 移动监督是专业用户默认形态
5. Agent 编排 agent (Sessions MCP 等)

1. 从痛点出发，场景压到 3 个以内——MVP 是裁
2. 先选对形态，再堆功能
3. 为 AI 协作修路：doc · skills · worktree · 验收
4. 优先打磨信任相关体验（状态 / 性能 / 像素）
5. 保持删的勇气



产品 × 工程 × **AI 回路** = 从想法到喜爱

谢谢

Termio —the terminal home for your AI coding agents

<https://termio.sh>

Q & A